

Developer: **Ayush Kumar** • [LinkedIn Profile](#)

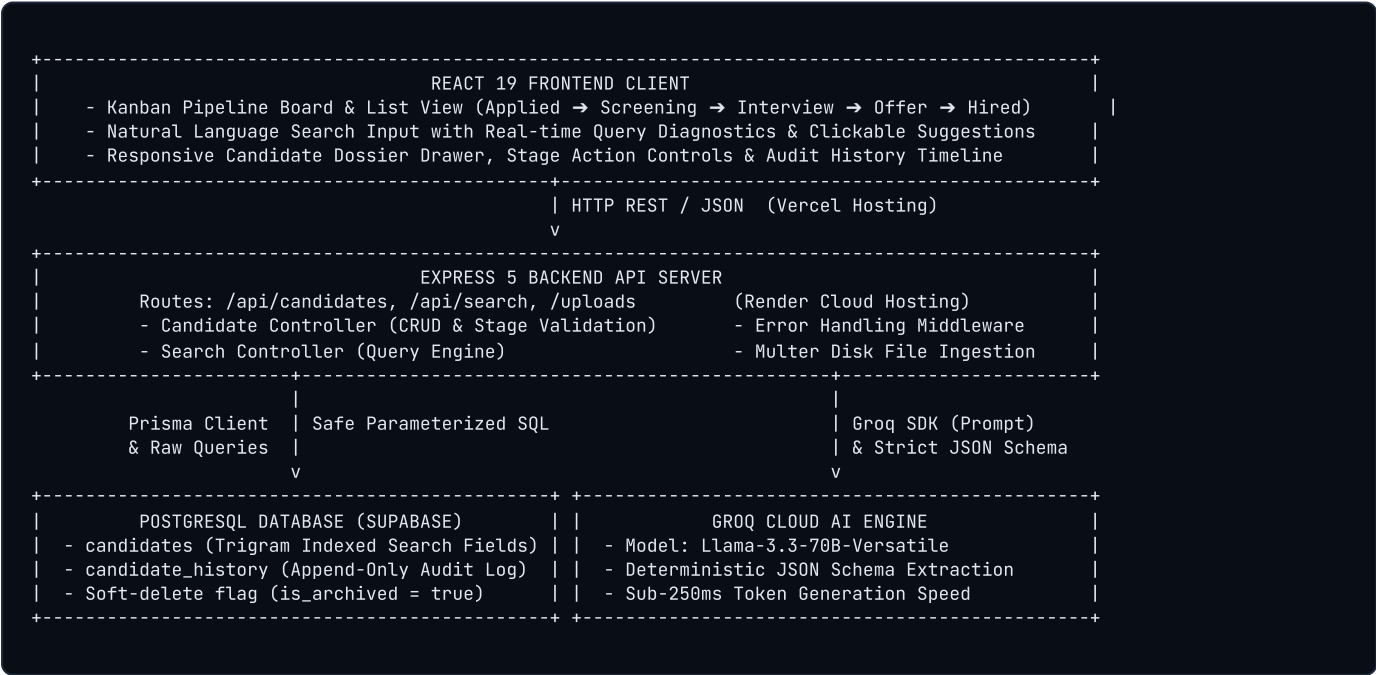
1. TECHNOLOGY STACK ARCHITECTURE

The platform is constructed with a decoupled micro-monorepo design. Each technology in the stack was selected to guarantee zero runtime bloat, lightning-fast inference, and database-level audit compliance.

Layer	Technology & Version	Architectural Role
Frontend Client	React 19 • TypeScript • Vite 8 • Lucide React	Interactive pipeline interface featuring both Kanban and List views, instant stage movements, search debounce throttling, and responsive candidate dossier drawer.
Styling & Design	Pure Vanilla CSS Design System (880+ lines)	Handcrafted fluid CSS grid, responsive layout tokens, and granular breakpoints supporting desktop dashboards, tablets, and ultra-narrow mobile viewports without Tailwind bloat.
Backend API Server	Node.js • Express 5 • TypeScript • Multer	REST API controllers with robust error-handling middleware, multipart file streaming for CVs/resumes, and business logic pipeline enforcement.
Database & ORM	PostgreSQL 15 (Supabase) • Prisma ORM 5	Connection pooling via PgBouncer with dedicated transaction and session poolers. Direct trigram similarity indexes (pg_trgm) for fuzzy search.
NLP Query Engine	Groq Cloud SDK • Llama-3.3-70B-Versatile	Transforms conversational recruiter queries into typed, deterministic JSON filter schemas with ultra-low token generation latency (<250ms).

2. HIGH-LEVEL SYSTEM ARCHITECTURE DIAGRAM

The architecture operates on a unidirectional request-response flow ensuring all natural language requests are translated into deterministic, type-safe SQL queries before touching the database:



3. CORE ARCHITECTURAL MODULES & IMPLEMENTATION DETAILS

1. Pipeline State Machine Workflow

Enforces strict forward-only stage transitions: Applied → Screening → Interview → Offer → Hired. Candidates cannot skip stages or revert from terminal stages (Hired/Rejected). Backward status corruption is prevented at both the frontend UI and backend transaction levels.

2. Immutable Audit Trail (CandidateHistory)

Every promotion, rejection, or feedback note appends an immutable record to candidate_history with onDelete: NoAction. No update() or delete() methods exist on this model, guaranteeing unalterable regulatory compliance.

3. Groq Natural Language Query Engine

Conversational queries such as "Candidates with 5+ years experience in React stuck in interview for more than a week" are interpreted by Groq Llama-3.3-70B into structured filter objects with numeric thresholds, array containment, and stage criteria.

4. Parameterized Search & Ranking

Parsed JSON filters are mapped into safe Prisma WHERE clauses and raw SQL trigram similarity queries (similarity(name, query) > 0.25). Results are scored, sorted by relevance, and accompanied by transparent match explanations.

4. KEY ARCHITECTURAL DECISIONS & ENGINEERING TRADE-OFFS

Architectural Decision	Alternative Considered	Engineering Rationale
Groq Llama-3.3-70B Query Engine	Client/Server regex heuristics, standard Gemini Flash, or local Python spaCy	Low Latency & High Reasoning Delivers sub-250ms extraction speed without requiring a heavy Python ML runtime. Avoids brittle regex while capturing nuanced, colloquial recruiter search patterns.
Dedicated Append-Only History Table	Single updated_at timestamp or mutable status column on Candidate	Audit Integrity & Compliance Hiring workflows require verifiable audit logs. Storing duration per stage and recruiter feedback in an unalterable table protects recruitment legal defensibility.
Handcrafted Vanilla CSS over Tailwind	Tailwind CSS / Component UI Libraries (MUI/AntD)	Zero Bloat & Precision Enables micro-engineered fluid breakpoints and responsive card mechanics tailored for dual-screen foldables and mobile screens with zero framework overhead.
Disk Storage for Resumes via Multer	PostgreSQL BYTEA / Base64 DB blobs	Database Performance Storing binary PDFs in PostgreSQL causes rapid database bloat and slows down backups. Dedicated disk storage served statically via Express delivers fast streaming.

5. AI COLLABORATION & TECHNICAL DISAGREEMENT CASE STUDY

Technical Disagreement: Rule-Based Offline Regex vs. Pure LLM Intent Parsing

The AI Assistant's Proposal: During search implementation, the AI initially proposed a dual-tier approach featuring "Step 1: Offline Rule-Based NLP" using regex patterns (e.g. /(applied|screening|interview|offer|hired)/i, /(\d+)\+?\s*years?/i) before delegating to the LLM.

"Don't want this Step 1: Offline Rule-Based NLP. Remove the changes of step 1 only." — Engineer Direction

Why the Engineer Disagreed & Overruled the Proposal:

- **Brittle Semantic Interpretation:** Natural recruiter questions like "candidates who did screening in frontend" easily fool keyword matching (mistaking "screening" as candidate notes instead of stage status).
- **Codebase & Maintenance Duplication:** Maintaining an offline regex grammar alongside an LLM adds duplicate logic, conflicting edge cases, and high testing overhead.
- **Inference Speed Reality:** Because Groq executes Llama-3.3-70B in <250ms, a fragile offline regex fallback was completely redundant. Pure LLM extraction with deterministic schema output is cleaner and far more resilient.
- Full interaction records and prompt transcripts: [AI_CHAT_LOGS/README.md](#)

6. SECURITY, RELIABILITY & DATA INTEGRITY

SQL Injection Prevention

All database queries are executed via Prisma ORM parameterized methods or tagged template literals (`prisma.$queryRaw`...``). Raw, unescaped string concatenations are strictly forbidden across the codebase.

Soft Deletion Safeguards

Candidates are never hard-deleted from PostgreSQL. The `isArchived = true` flag prevents active pipeline clutter while permanently retaining candidate contact records and audit history.

CORS & API Environment Isolation

Dynamic CORS policy permits authorized client domains. Sensitive database credentials and Groq API tokens are stored in environment variables, never committed to version control.

Transaction-Atomic Stage Moves

Every candidate status update is wrapped in a PostgreSQL database transaction (`prisma.$transaction`), ensuring that candidate updates and audit history inserts either both succeed or both roll back.

7. FUTURE ROADMAP & WHAT WE'D BUILD WITH MORE TIME

If given additional development cycles, the platform would be enhanced with the following enterprise capabilities:

Feature Milestone	Proposed Implementation	Business & Technical Value
1. Hybrid Vector Search	pgvector extension in PostgreSQL + text-embedding-3-small	Enables semantic similarity matching across candidate CVs and notes (e.g. <i>"Find candidates with experience similar to a Senior DevOps Architect"</i>).
2. Automated Resume OCR	Background worker with PDF text extractor + LLM entity parsing	Automatically extracts skills, experience years, phone numbers, and past employers upon CV upload, eliminating manual data entry.
3. Real-Time Collaborative WebSockets	Server-Sent Events (SSE) or Socket.io integration	Allows hiring managers and interviewers to view pipeline movements and candidate advancements concurrently without manual browser refreshes.
4. Enterprise Multi-Tenant RBAC	JWT authentication with role guards (Admin, Recruiter, Reviewer)	Ensures departmental candidate access control, private salary feedback visibility, and recruitment compliance governance.
5. Calendar Scheduling Integrations	Google Calendar / Microsoft Outlook Webhooks	Enables one-click interview slot dispatching and automated candidate interview reminder notifications.

Assessment Summary & Verification:

All core platform requirements—including sequential pipeline stage transitions, immutable audit history logs, Groq-powered natural language candidate search, responsive UI design, and cloud deployments on Vercel and Render—are fully implemented, tested, and operational.